



A common software framework for multi-function radar via three-level dependency decoupling

Jinsoo Park^{a,b}, Heejeong Hong^b, Ikjun Yeom^a, Jinkyu Lee^{c,*}

^a Department of Computer Science and Engineering, Sungkyunkwan University, Republic of Korea

^b Radar R&D Center, Hanwha Systems Co., Ltd., Republic of Korea

^c Department of Computer Science and Engineering, Yonsei University, Republic of Korea

ARTICLE INFO

Keywords:

Multi-function radar
Software product line
Common software framework
Layered architecture
Intra-process communication

ABSTRACT

Modern multi-function radar (MFR) systems execute heterogeneous missions across diverse platforms, demanding a common software framework that enables systematic reuse across the product family. However, prevailing defense software practices remain anchored to monolithic architectures in which reusable core assets are inseparably entangled with volatile, project-specific modules. Transforming such codebases into a reusable framework has remained elusive due to three mutually reinforcing challenges: the absence of a quantitative basis to govern inter-module dependencies at the macro level, compile-time static I/O binding at the boundary level, and mutex-based synchronization at the micro level. We propose a framework that resolves inter-module coupling across three levels through three synergistic strategies. First, a volatility-dependency-based layered architecture enforces a unidirectional dependency flow that isolates invariant core assets from volatile modifications. Second, a protocol-agnostic dynamic I/O interface driven by an inversion-of-control mechanism achieves zero-code adaptability across heterogeneous environments. Third, RiPPS, a lightweight intra-process publish-subscribe middleware, guarantees anonymous communication and deadlock-free concurrency via a memory-isolated stream. Evaluation on two operational MFR subsystems demonstrates reductions of up to 51% in modified cyclomatic complexity, 42% in coupling references, and 56% in lines of code, while concurrency benchmarks confirm that RiPPS preserves publish-subscribe anonymity at latencies competitive with bare lock-free queues.

1. Introduction

Modern multi-function radar (MFR) systems powered by active electronically scanned array technology have been fundamentally reshaping the defense radar landscape. Unlike legacy mechanical radars dedicated to a single mission, MFRs concurrently execute heterogeneous tasks — such as volumetric search, precision tracking, and missile guidance — while adapting to platform-specific hardware across land-based surveillance, naval combat, and airborne fire control [1]. This operational versatility gives rise to a diverse product family of radar variants. While each variant is configured for a distinct platform and threat environment, this growing diversity has intensified the need for a shared, reusable radar software baseline that can serve the entire product family. Consequently, radar software engineering practices must transition from developing monolithic software bound to a single product toward constructing a common software framework that enables systematic reuse across the entire product line.

However, prevailing software engineering practices in the defense sector remain fundamentally misaligned with this product-line demand.

Conventional systems adhere to a monolithic architecture strictly tailored to a single project's requirements. In such unmanaged structures, structurally stable, reusable core assets that should remain invariant across projects are tightly intertwined with highly volatile, project-specific hardware drivers and communication protocols. Because established module decomposition criteria provide no mechanism to distinguish these two orthogonal architectural dimensions — the *volatility* of a module, which captures how extensively its implementation must be modified when reused in a different project, and its *dependency*, which captures how structurally dependent the module's position is within a single project — this entanglement produces two compounding consequences. First, within a single project, even a localized modification to a low-level hardware interface triggers unpredictable ripple effects throughout the core mission algorithms, substantially increasing the cost of maintenance and reverification. Second, across different projects, extracting a verified software module for reuse in a new

* Corresponding author.

E-mail addresses: waldo0322@skku.edu (J. Park), heejeong99@hanwha.com (H. Hong), ikjun@skku.edu (I. Yeom), jinkyu.lee@yonsei.ac.kr (J. Lee).

<https://doi.org/10.1016/j.sysarc.2026.104021>

Received 22 April 2026; Received in revised form 9 August 2026; Accepted 19 September 2026

Available online 24 September 2026

1383-7621/© 2026 Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

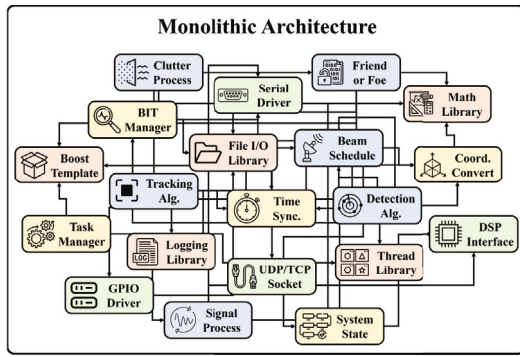


Fig. 1. Legacy monolithic architecture where all modules coexist in a single flattened layer with uncontrolled bidirectional dependencies.

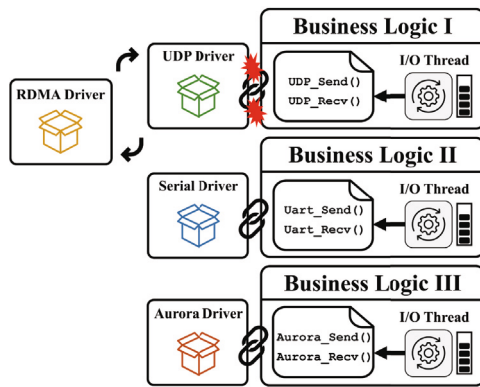


Fig. 2. Legacy I/O structure where business logic directly references concrete protocol implementations, forcing source modification upon any transport change.

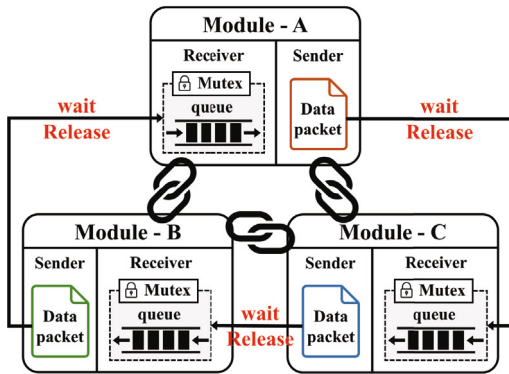


Fig. 3. Legacy concurrency model where direct mutex-based inter-module coupling creates an $M \times N$ dependency mesh, increasing deadlock risks as 1:N and M:1 patterns grow.

radar variant is practically infeasible because the target module remains coupled to project-specific components from which it cannot be independently separated.

Both consequences stem from a common root cause: the absence of systematic architectural mechanisms to govern inter-module dependencies and decouple highly volatile components from structurally stable components. Without such mechanisms, modules remain inseparably coupled, impeding both efficient maintenance within a project and reliable asset reuse across projects. This architectural deficiency manifests at three distinct levels of the software structure—from the *macro-level* system decomposition, through the *boundary-level* external

device interface, to the *micro-level* intra-process communication. Transforming a legacy monolithic system into a scalable, reusable common software framework therefore requires addressing three interconnected architectural challenges, referred to as C1–C3. We analyze these not merely as implementation hurdles, but as fundamental architectural gaps.

- C1. *Macro-level: structural complexity and limited reusability from the absence of a dependency-governed architectural baseline.* As Fig. 1 illustrates, legacy MFR modules coexist within a single flattened layer that provides no quantitative basis to classify modules by their volatility or dependency, leaving inter-module dependency directions uncontrolled and reusable assets indistinguishable from project-specific code. Open architecture standards such as the modular open systems approach (MOSA) [2] and the sensor open systems architecture (SOSA) [3] standardize external subsystem interfaces but treat internal software structure as a black box. In contrast, software product line engineering (SPLE) [4,5] manages feature-level variability and entrusts the structural organization of the underlying code modules to the product-line architect—a responsibility that calls for quantitatively classifying modules by volatility and dependency for structural isolation.
- C2. *Boundary-level: adaptability constraints from compile-time static I/O binding.* In legacy MFR systems, as depicted in Fig. 2, business logic directly references concrete protocol drivers, and this binding is resolved at compile time through conditional compilation directives, forcing invasive source modification and full recompilation whenever the deployment platform or testing environment changes. Conventional hardware abstraction layer (HAL) implementations [6] in embedded domains reinforce this static binding practice; modern embedded ecosystems rebind their abstractions only inside full middleware runtimes [7,8]; and mainstream inversion of control (IoC) frameworks [9] presuppose managed runtime environments and heavyweight infrastructure that are incompatible with resource-constrained embedded platforms.
- C3. *Micro-level: tight coupling and concurrency hazards in intra-process communication.* Legacy mutex-based module synchronization, as illustrated in Fig. 3, forces direct inter-module references that undermine any hierarchical decomposition at the communication level, while the resulting lock-ordering complexity introduces escalating risks of deadlocks and priority inversions. Standard data distribution service (DDS) [10] provides anonymous topic-based communication but imposes prohibitive network stack overhead for intra-process deployment [11–13], whereas lock-free queues [14] eliminate blocking but retain direct inter-module references without publish–subscribe routing.

Critically, these challenges form a mutually reinforcing cycle of structural degradation: a solution at any single level remains ineffective if the remaining levels continue to impose uncontrolled coupling. This necessitates an integrated architectural response that addresses dependencies across all three levels simultaneously.

In this paper, we propose a common software framework for MFR systems that addresses C1–C3 through three synergistic architectural strategies, thereby fulfilling the structural responsibility that SPLE entrusts to the product-line architect. Each strategy targets a distinct challenge level: a volatility-dependency-based layered architecture that establishes macro-level structural boundaries (for C1), a protocol-agnostic dynamic I/O interface that enables boundary-level runtime adaptability (for C2), and RiPPS, a radar intra-process publish–subscribe system that enforces micro-level inter-module communication decoupling (for C3).

To address C1, we propose a quantitative restructuring strategy that classifies legacy modules using two orthogonal metrics — volatility and dependency — and segregates the system into three layers: framework, platform, and application. The resulting layered architecture enforces

the stable dependencies principle (SDP), restricting dependency flow to a strict top-down direction so that frequent changes in the upper application layer never propagate to the reusable core assets in the lower layers.

For C2, we implement an IoC container that dynamically instantiates and injects protocol-specific communication objects based on external runtime configurations. This mechanism strictly enforces the dependency inversion principle (DIP) [15], enabling zero-code adaptability where transport changes — including transitions between live hardware and digital twin simulation environments — are absorbed entirely by configuration updates without any source modification or recompilation.

To resolve C3, we introduce RiPPS, an architectural decoupling middleware for intra-process communication. RiPPS establishes an anonymous topic-based backbone that completely eliminates direct inter-module references, and employs a memory-isolated mechanism that guarantees deadlock-free concurrency safety by design rather than by manual lock-ordering discipline.

We evaluate the proposed framework by independently restructuring each of two operational MFR subsystems — both extracted from a deployed phased array radar — using all three architectural strategies. For each subsystem, we measure the structural improvement from its legacy monolithic implementation to its restructured baseline. The results demonstrate that the framework achieves reductions of up to 51% in modified cyclomatic complexity (MCC), 42% in total coupling references (TCR), and 56% in lines of code (LOC), substantiating that the proposed restructuring yields measurable improvements in maintainability and structural soundness. In addition, comparative concurrency benchmarks confirm that RiPPS uniquely guarantees anonymous decoupling and structurally deadlock-free concurrency, thereby satisfying both architectural independence and real-time safety within a single mechanism.

In summary, this paper makes the following contributions:

- We identify three mutually reinforcing architectural challenges in legacy MFR software — at the macro, boundary, and micro levels — and establish the necessity of an integrated decoupling strategy (Section 2).
- We propose a volatility-dependency-based layered architecture that enforces unidirectional dependency flow across three quantitatively defined layers (Section 3).
- We implement a protocol-agnostic dynamic I/O interface that achieves zero-code adaptability through configuration-driven protocol binding (Section 4).
- We introduce RiPPS, a lightweight intra-process publish-subscribe middleware that guarantees anonymous communication and deadlock-free concurrency (Section 5).
- We evaluate the framework on two operational MFR subsystems, demonstrating measurable reductions in complexity, coupling, and code volume, together with comparative concurrency benchmarks (Section 6).

The remainder of this paper is organized as follows. Section 2 traces each challenge to its root cause in a representative legacy MFR system. Sections 3–5 develop the three architectural strategies that resolve C1–C3, respectively. Section 6 presents the structural improvements and concurrency benchmarks. Section 7 reviews related work; Section 8 discusses the costs and limitations of the framework; and Section 9 concludes the paper.

2. Architectural challenges in legacy MFR systems

This section examines the three architectural challenges introduced in Section 1 through the structural analysis of a representative legacy MFR system. For each challenge, we identify the root cause, characterize the resulting engineering constraint, and derive the design objective that a principled solution must satisfy.

2.1. Macro-level: monolithic architecture (C1)

Fig. 1 illustrates a representative legacy MFR system in which all modules — from domain-agnostic mathematical libraries and hardware interface adapters to mission-critical tracking algorithms — coexist within a single execution layer with no hierarchical separation. Such a non-hierarchical layout provides no quantitative basis to classify modules by their volatility and dependency. As a result, inter-module dependency directions are entirely uncontrolled; a reusable core asset may reference a volatile hardware driver, or vice versa, resulting in unconstrained bidirectional coupling. Under these conditions, modifications to any single module can propagate along arbitrary reference paths, making the scope of change unpredictable and rendering reverification prohibitively expensive. The problem is further compounded when reuse is attempted across projects. Because all modules reside in the same undifferentiated layer, there is no principled way to determine which modules constitute project-invariant assets and which are inherently project-specific. Furthermore, even an intuitively identified reusable candidate cannot be independently extracted because it shares entangled dependencies with surrounding project-specific components. The challenge is therefore to establish a rigorous architectural boundary — governed not by subjective functional categorization, but by quantitative decomposition metrics — that structurally isolates reusable core assets and enforces unidirectional dependency flow by design.

2.2. Boundary-level: static I/O binding (C2)

MFR systems must interface with a heterogeneous set of external devices through diverse communication protocols — UDP, TCP, RDMA, serial, and others — that vary by deployment platform and hardware revision. In embedded defense systems, the dominant approach to bridging business logic and these physical transports is compile-time static binding, such as conditional compilation and preprocessor directives, primarily to eliminate runtime overhead and maximize deterministic execution, as illustrated in Fig. 2. While this approach is effective for a fixed hardware configuration, it creates a fundamental adaptability constraint once the system must accommodate multiple configurations. Because the business logic directly references concrete protocol implementations rather than abstractions, any change in the underlying transport — whether replacing UDP with RDMA for a new sensor topology or switching from hardware-bound protocols to standard LAN interfaces for software-based digital twin simulation — forces invasive modifications to the application source code and necessitates a full recompilation. This rigidity directly violates the open-closed principle (OCP) [15]. The challenge is therefore to achieve configuration-driven protocol binding in a resource-constrained embedded environment. The objective is to enable zero-code adaptability — where transport changes are absorbed entirely by configuration rather than source modification — without incurring the overhead penalties that motivated static binding in the first place.

2.3. Micro-level: intra-process communication (C3)

As MFRs evolve into multi-mission platforms, the internal data exchange patterns among software modules grow correspondingly complex. A single beam-scheduling decision, for instance, must be distributed to multiple downstream consumers such as waveform generators, signal processors, and display renderers in a 1:N fan-out pattern, while the tracking engine must aggregate measurement reports from multiple sensor channels in an M:1 fan-in pattern. Legacy systems handle these exchanges through direct module-to-module synchronization using OS primitives such as mutexes and shared queues, as illustrated in Fig. 3. In this model, a sending module must hold an explicit reference to every receiving module's queue, and vice versa, creating an $M \times N$ mesh of point-to-point dependencies among communicating modules.

This direct-reference coupling poses a critical architectural problem: no matter how rigorously the system is decomposed into hierarchical layers at the macro level, the resulting layer boundaries are nullified if the internal communication still forces modules to know each other's identity and memory layout.

Beyond structural concerns, the safety implications of this approach escalate non-linearly with communication complexity. Each new data path introduces additional mutexes and lock-ordering constraints that must be manually coordinated across the entire system. As the number of 1:N and M:1 patterns grows, the combinatorial explosion of potential lock-acquisition sequences makes it increasingly difficult to guarantee freedom from deadlocks and to prevent priority inversions, where a high-priority radar control task is blocked by a lower-priority logging task holding a contested lock. The challenge is therefore twofold: first, to eliminate direct inter-module references by introducing an anonymous communication backbone that preserves hierarchical isolation at the intra-process level; and second, to provide a concurrency mechanism that is structurally immune to deadlocks and priority inversions by design, rather than by manual lock-ordering discipline.

3. V-D-based layered architecture

In this section, we propose a quantitative restructuring strategy that addresses the macro-level structural complexity and limited reusability inherent in legacy monolithic architectures (C1) by replacing subjective functional categorization with a metric-driven decomposition. We first present the methodology for applying the framework to a legacy product family. We then formalize the two orthogonal engineering metrics on which it rests — volatility (V) and dependency (D) — and apply these metrics to segregate the system into three distinct layers — framework, platform, and application — whose dependency directions are governed by SDP to enforce a strict unidirectional flow, thereby establishing the dependency-governed architectural boundaries.

3.1. Framework application methodology

Applying the proposed framework to a legacy product family proceeds as follows. Since volatility is observable only through cross-project comparison, we begin by declaring an analysis scope $\mathcal{P}$, the set of existing radar variants within the product family targeted by the common software framework. The reliability of V therefore rests less on the number of variants in $\mathcal{P}$ than on how well they cover the major factors inducing volatility, namely shifts in the mission operational concept — from air surveillance to ballistic missile defense — and in the deployment environment. For a framework targeting land-based, naval, and airborne radars, for instance, the scope must include variants from each platform. With the scope declared, we measure the volatility V of each module through a line-level comparison of the variants' source snapshots, which separates the lines surviving verbatim in every variant from the remaining lines of its counterpart in each variant; averaging the resulting per-variant modification ratios yields V , as Eqs. (1) and (2) formalize. We likewise extract the dependency D of each module through static analysis of the legacy system under restructuring, counting the modules it references and those referencing it, as Eq. (3) formalizes. We then map the modules onto a coordinate plane defined by V and D and assign each to one of the three layers, as Section 3.3 details.

The classification, however, prescribes only the target layer of each module; conforming the legacy codebase to this structure requires refactoring, which the preceding measurements guide in two ways. First, a module whose lines interleave invariant logic with project-specific logic resists assignment to a single layer. Because the line-level comparison identifies which lines survive in every variant and which do not, we split such a module along this boundary and re-measure the parts; otherwise, the project-specific fraction would exclude the entire module, including its invariant core, from the reusable lower layers.

Second, layer assignment exposes the legacy dependencies that violate the unidirectional flow, such as business logic directly referencing concrete protocol drivers or modules holding explicit references to each other's queues; replacing the former with an abstract interface and the latter with anonymous communication is precisely the role of the mechanisms presented in Sections 4 and 5, respectively. Each legacy system adopting the common framework therefore undergoes this conversion once, after which new variants derive directly from the restructured baseline.

Taken together, the declared scope, the line-level measurement of V , the static extraction of D , and the measurement-guided refactoring constitute the methodology for applying the framework.

3.2. Quantitative decomposition metrics

To systematically classify legacy modules without relying on subjective functional boundaries, we define two orthogonal metrics that quantify the cross-project volatility and the intra-project dependency of each module.

The volatility metric V quantifies the cross-project volatility of a module, measuring how extensively its implementation must be modified when reused in a different radar project. Given a module, each variant p_i in the subset $\mathcal{P}_m \subseteq \mathcal{P}$ of variants containing it yields one modification ratio v_i , as defined in Eq. (1):

$$v_i = \frac{\text{LOC}_{\text{variable},i}}{\text{LOC}_{\text{fixed}} + \text{LOC}_{\text{variable},i}} \quad (1)$$

where $\text{LOC}_{\text{fixed}}$ denotes the number of the module's lines that survive verbatim in every variant of $\mathcal{P}$, and $\text{LOC}_{\text{variable},i}$ the number of remaining lines of its counterpart in variant p_i ; a line-level comparison of the variants' source snapshots determines both counts without manual estimation. The volatility of the module is then the average of these ratios over $\mathcal{P}_m$, as given in Eq. (2):

$$V = \frac{1}{|\mathcal{P}_m|} \sum_{p_i \in \mathcal{P}_m} v_i \quad (2)$$

Because $\text{LOC}_{\text{fixed}}$ counts only the lines surviving in every variant, a module absent from at least one variant of $\mathcal{P}$ — including a module introduced by a single project — has $\text{LOC}_{\text{fixed}} = 0$ and receives $V = 1$. A sonar interface adapter shared verbatim across naval variants, for instance, has $V = 0$ under a naval-only scope yet $V = 1$ under an all-platform scope; volatility is thus meaningful only relative to the chosen scope. A module with a low V value encapsulates domain-agnostic logic that remains invariant across different radar projects, representing the reusable core assets of the product line. In contrast, a module with a high V value implements project-specific requirements or hardware-dependent logic, necessitating significant rewrites for each deployment.

The dependency metric D quantifies the intra-project dependency of a module, capturing how structurally dependent the module's position is within a single radar project. We accordingly evaluate D on the legacy system under restructuring. Adapting the instability metric proposed by Martin [16], we define D as the ratio of outgoing dependencies to total dependencies, as formulated in Eq. (3):

$$D = \frac{\text{Fan}_{\text{out}}}{\text{Fan}_{\text{in}} + \text{Fan}_{\text{out}}} \quad (3)$$

where Fan_{out} denotes the number of external modules referenced by the target module, and Fan_{in} represents the number of modules referencing the target, both extracted through static analysis of the legacy codebase. A module with a low D value approaching zero functions as an independent provider; it is structurally stable because it delivers functionality without relying on underlying modules. Conversely, a module with a high D value approaching one acts as a dependent consumer. It is structurally unstable because its execution relies heavily on numerous lower-level modules, making it inherently vulnerable to the ripple effects triggered by localized modifications in any of its dependencies.

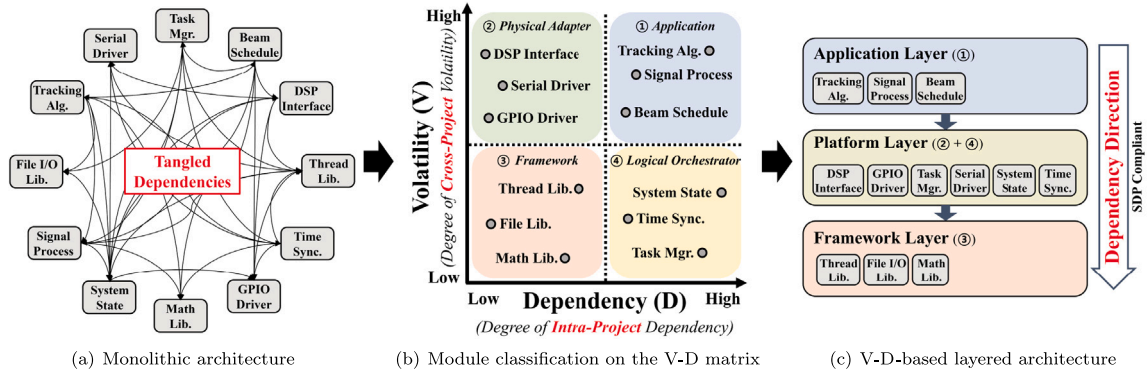


Fig. 4. Transition from legacy monolithic architecture to proposed V - D -based layered architecture.

3.3. Dependency-governed layered decomposition

By mapping the legacy modules of Fig. 4(a) onto the coordinate plane defined by V and D , we identified four distinct clusters, as shown in Fig. 4(b). Based on their architectural characteristics and roles within the radar domain, we consolidated these clusters into three structural layers — framework, platform, and application — yielding the dependency-governed layered architecture in Fig. 4(c).

The framework layer comprises modules residing in the low V and low D region. This layer serves as the invariant foundation of the radar software product line. Operating as domain-agnostic utilities, these modules are entirely independent of specific radar hardware configurations or mission requirements. Since they neither change across projects nor depend on system-specific logic, they constitute the reusable core assets shared across the entire product line. Representative examples include linear algebra engines, standard template library extensions, file I/O utilities, and high-performance logging libraries.

The platform layer occupies the intermediate region of the V - D coordinate space, serving as the unified execution context that isolates the upper application layer from the complexities of both hardware environments and system-level coordination. The design principle of this layer is *context encapsulation*: by absorbing these two sources of complexity into a single architectural stratum, the platform layer establishes an isolation boundary that enables the application layer to focus exclusively on mission-specific business logic. Although this layer encompasses two metrically distinct module clusters on the V - D plane — physical adapters in the high V , low D region and logical orchestrators in the low V , high D region — their consolidation into one layer is a deliberate design decision grounded in their complementary roles in achieving this encapsulation.

Physical adapters interact directly with hardware interfaces, encapsulating the environmental volatility introduced by platform-specific peripherals. A representative example is a hardware-specific sensor interface driver. Its internal implementation must be substantially rewritten whenever the deployment platform changes, inherently yielding a high V value. However, it relies solely on the domain-agnostic utilities of the framework layer and references no application-level modules, establishing a strictly low D value. Its architectural contribution to context encapsulation lies in shielding the rest of the system from this hardware-induced volatility. While this implementation is highly project-specific, the abstract interface contract it exposes to the upper layers constitutes a reusable, invariant asset across all radar variants.

Logical orchestrators coordinate system-wide execution by managing the operational lifecycle, resource allocation, and inter-subsystem command sequences of the radar system. A representative example is the system lifecycle manager, which orchestrates power-up sequences, built-in test (BIT) procedures, and operational mode transitions across the entire radar topology. The state-machine logic governing these system-level transitions remains fundamentally invariant across different radar variants — whether land-based air defense or naval combat

systems — yielding a low V value. To execute these control sequences, however, the lifecycle manager must reference and issue commands to nearly all subordinate physical adapters and framework utilities, driving a high D value. Despite this high structural dependency, the orchestration logic itself constitutes a highly reusable cross-project asset; its core algorithms transfer across projects with minimal modification, and only the specific set of subordinate modules it coordinates varies with each deployment configuration.

The application layer consists of modules residing in the high V and high D region, representing the mission-specific implementation of each radar variant. These modules are inherently volatile due to frequent changes in operational requirements, such as engagement doctrines or threat-specific search patterns. They are also highly dependent, as they compose the domain-enriched services provided by the platform layer to execute complex business logic. Representative examples include beam scheduling algorithms and target tracking filters tailored to specific tactical environments. By isolating this high-volatility, high-dependency logic at the topmost layer, engineers can rapidly iterate on mission algorithms, guaranteeing that the isolation boundary of the platform layer structurally confines these volatile changes.

To maintain the structural integrity of this layered decomposition, we enforce SDP through the strict unidirectional dependency flow depicted in Fig. 4(c): the application layer depends on the platform layer, which in turn depends on the framework layer. No reverse or bypass dependencies are permitted. This architectural enforcement serves a dual purpose, directly resolving the two consequences of C1 identified in Section 1. First, within a single project, it eliminates the ripple effects of localized modifications. Because dependency flows exclusively downward, changes in the highly volatile application layer cannot propagate into the stable platform and framework layers, thereby preserving the integrity of the reusable core assets upon which the entire system relies. Second, across different projects, it guarantees the independent extractability of verified assets. Because the lower layers carry no references to the application layer, the framework and platform modules can be extracted and deployed in a new radar variant without pulling in project-specific dependencies—a structural impossibility under the legacy monolithic architecture, where unconstrained bidirectional coupling entangled all modules into an inseparable whole.

This classification-driven restructuring ultimately resolves C1 by transforming an architecturally ungoverned codebase into a dependency-governed layered structure that supports both efficient intra-project maintenance and reliable cross-project reuse.

4. Protocol-agnostic dynamic I/O interface

Building upon the macro-level layered architecture established in Section 3, this section addresses the boundary-level adaptability constraints caused by compile-time static I/O binding (C2), as depicted

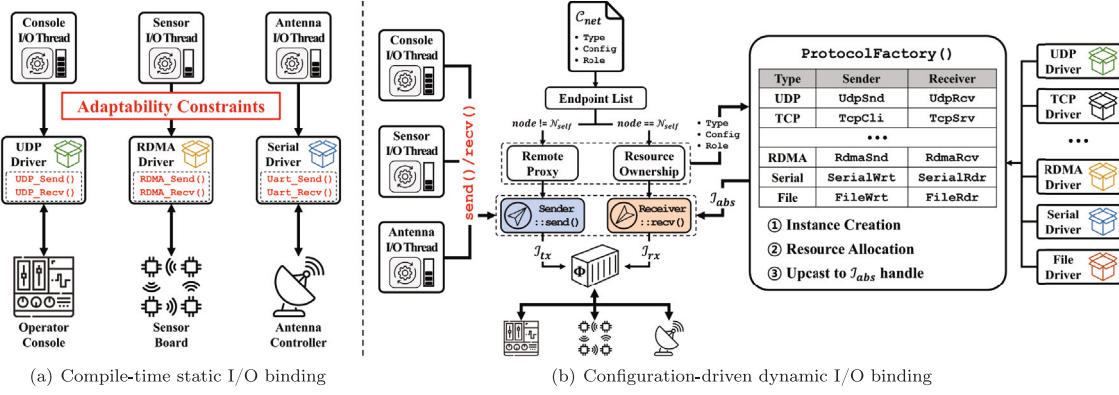


Fig. 5. Transition from legacy compile-time static I/O binding to configuration-driven dynamic I/O binding.

Algorithm 1 Topology-driven orchestration

```

1: Input:  $C_{net}$  (Network config),  $N_{self}$  (Current node ID)
2: Output:  $\Phi$  (IoC container)
3:  $\Phi \leftarrow \text{CreateEmptyContainer}()$ 
4:  $\mathcal{E}_{list} \leftarrow \text{ParseTopology}(C_{net})$ 
5: for all endpoint  $e \in \mathcal{E}_{list}$  do
6:    $Type \leftarrow e.protocol$  // e.g., UDP, RDMA, Serial
7:    $Config \leftarrow e.settings$  // e.g., IP:Port, device path
8:   if  $e.node == N_{self}$  then
9:     // Case 1: Inbound role (Resource ownership)
10:     $I_{rx} \leftarrow \text{ProtocolFactory}(Type, Config, Role::Receiver)$ 
11:     $\Phi.RegisterReceiver(e.name, I_{rx})$ 
12:   else
13:     // Case 2: Outbound role (Remote proxy)
14:     $I_{tx} \leftarrow \text{ProtocolFactory}(Type, Config, Role::Sender)$ 
15:     $\Phi.RegisterSender(e.name, I_{tx})$ 
16:   end if
17: end for
18: return  $\Phi$ 

```

in Fig. 5(a). We first define a unified abstract interface that decouples business logic from concrete protocol implementations through dependency inversion. We then introduce an IoC-based orchestrator paired with a polymorphic factory that dynamically instantiates and injects the appropriate protocol adapters at system initialization, driven by an external network topology descriptor. Together, these mechanisms achieve configuration-driven protocol binding that enables zero-code adaptability across heterogeneous deployment environments.

4.1. Dependency inversion at the I/O boundary

The root cause of this adaptability constraint is a missing abstraction boundary: without an intermediate interface that separates *what* the application requests from *how* the transport delivers it, every protocol change propagates upward into business logic. We resolve this by applying DIP to invert the dependency direction at the external communication boundary. Specifically, we interpose a unified abstract interface, denoted as I_{abs} , that both sides of this boundary now depend upon: the application layer references only I_{abs} for all data exchange, while each concrete protocol adapter — such as UDP, RDMA, serial links, and others — implements I_{abs} from below. This inversion ensures that the dependency arrow points toward the abstraction rather than any concrete implementation, as shown in Fig. 5(b).

The data-path contract of I_{abs} exposes only two protocol-agnostic primitives, specifically `send()` and `receive()`, while encapsulating protocol-specific setup operations, such as network port binding and memory region registration, internally within each adapter. The

Algorithm 2 Polymorphic protocol factory

```

1: Input:  $Type, Config, Role$ 
2: Output:  $I_{abs}$  (Abstract interface)
3:  $Class \leftarrow null$  // Concrete adapter class
4: // 1. Select concrete class based on protocol type
5: if  $Type$  is "UDP" then
6:    $Class \leftarrow (Role == Sender)? \text{UdpSnd} : \text{UdpRcv}$ 
7: else if  $Type$  is "TCP" then
8:    $Class \leftarrow (Role == Sender)? \text{TcpCli} : \text{TcpSrv}$ 
9: else if  $Type$  is "RDMA" then
10:   $Class \leftarrow (Role == Sender)? \text{RdmaSnd} : \text{RdmaRcv}$ 
11: else if  $Type$  is "Serial" then
12:   $Class \leftarrow (Role == Sender)? \text{SerialWrt} : \text{SerialRdr}$ 
13: else if  $Type$  is "FILE" then
14:   $Class \leftarrow (Role == Sender)? \text{FileWrt} : \text{FileRdr}$ 
15: end if
16: // 2. Instantiate and upcast (DIP enforcement)
17: if  $Class \neq null$  then
18:   $Obj \leftarrow \text{new } Class(Config)$  // Inject config
19:   $Obj.AllocResource()$  // e.g., Bind, Connect, Open
20:   $I_{abs} \leftarrow \text{UpcastToAbstract}(Obj)$ 
21:  return  $I_{abs}$ 
22: end if
23: return  $null$ 

```

abstraction is deliberately minimal: it captures only the essential data-exchange contract shared by every external transport in the radar domain, encompassing connectionless datagram delivery over UDP, one-sided memory operations over RDMA, and byte-serial exchange over standard serial interfaces. By restricting the interface to this common denominator, we guarantee that protocol-specific details never leak into the application layer, regardless of how the underlying transport evolves.

Within the proposed layered architecture, I_{abs} and its concrete adapters reside in the platform layer as a subset of the physical adapters responsible for external communication. Because the application layer interacts exclusively with I_{abs} rather than any concrete adapter class, the system strictly enforces the unidirectional dependency flow prescribed by SDP at the boundary level. Consequently, introducing a new transport or replacing an existing one requires only the addition of a new adapter that implements I_{abs} , leaving the application layer source code entirely untouched and preserving the OCP.

4.2. Topology-driven orchestration and polymorphic protocol factory

Although the abstract interface I_{abs} establishes the necessary abstraction boundary, it does not dictate how the correct concrete adapter is selected and instantiated for each communication endpoint. We present two complementary mechanisms to resolve this instantiation

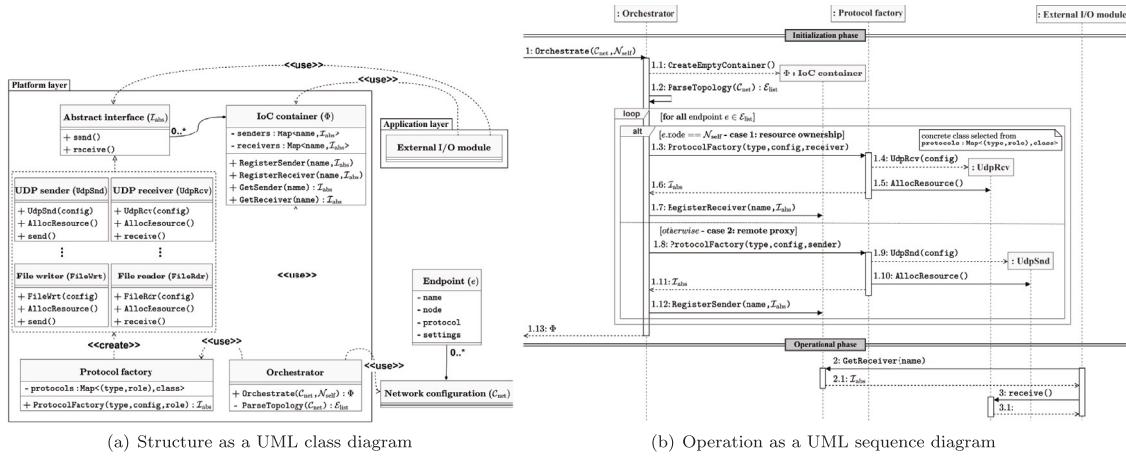


Fig. 6. Design of the protocol-agnostic dynamic I/O interface.

challenge. The first is an IoC-based orchestrator, outlined in Algorithm 1, which determines the required adapter for each endpoint based on an external topology descriptor. The second is a polymorphic factory, detailed in Algorithm 2, which performs the actual instantiation and returns the object exclusively through the I_{abs} handle. Because both mechanisms execute entirely during the system initialization phase, all protocol bindings are fully resolved before the first operational cycle, imposing no runtime overhead on the critical execution path. Beyond these two mechanisms and the container they populate, the design requires no additional infrastructure, remaining lightweight enough for resource-constrained embedded platforms.

Algorithm 1 details the orchestration logic executed during system initialization. The orchestrator first parses a network topology configuration C_{net} — an external descriptor that captures the identities, protocol types, and addressing parameters of all nodes in the target deployment — to generate a complete endpoint list $\mathcal{E}_{list}$ (Line 4). It then iterates through every endpoint and determines its role based on the current node identity $\mathcal{N}_{self}$ (Lines 5–8). A pivotal architectural decision in this orchestration is the distinction between two endpoint roles: *resource ownership* and *remote proxy*. When an endpoint belongs to the current node (Lines 9–11), the orchestrator assigns the resource ownership role. It invokes the factory to instantiate a receiver I_{rx} that actively binds to a local physical resource — such as opening a listening port for UDP/TCP, registering a memory access region for RDMA, or initializing a device handle for a serial interface — because the node must be prepared to accept inbound traffic on that endpoint. Conversely, if the endpoint belongs to a remote node (Lines 13–15), the orchestrator assigns the remote proxy role. It instantiates a sender I_{tx} that retains only the addressing information necessary to reach the remote endpoint, without occupying any local physical resource, since its sole responsibility is outbound delivery. Finally, the orchestrator registers all initialized instances into the IoC container Φ . The application layer subsequently retrieves these instances exclusively via the I_{abs} handle, ensuring complete abstraction.

Algorithm 2 presents the polymorphic factory invoked by the orchestrator. Given a protocol type, endpoint configuration, and communication role as input, the factory selects the corresponding concrete adapter class from its internal mapping table (Lines 5–15), instantiates the object with the endpoint-specific settings, and allocates the required transport resources (Lines 18–19). The pivotal operation is the upcast at Line 20: the factory casts the concrete object to I_{abs} before returning it to the orchestrator. This upcast enforces DIP at the instantiation boundary—once the concrete object is returned as an abstract handle, no protocol-specific type information escapes the factory, and the application layer remains entirely agnostic to the concrete class behind the handle. From a maintainability standpoint, the factory’s mapping table constitutes the single, well-isolated extension point of the entire

I/O subsystem: supporting a new protocol requires adding one mapping entry, with no modification to the orchestration logic, the application layer, or any existing module.

Fig. 6 presents the resulting design in two complementary aspects of a software architecture description [17]: the class diagram in Fig. 6(a) depicts its structure, and the sequence diagram in Fig. 6(b) its operation. Structurally, the concrete protocol adapters realize the Adapter pattern, each converting a heterogeneous underlying transport to the unified abstract interface, and the polymorphic factory realizes the parameterized form of the Factory Method pattern [18], in which the type and role parameters select the concrete class to instantiate. Fig. 6(a) also documents the internal structure of each participant, from the factory’s mapping table to the container’s name-indexed registries of abstract handles. In Fig. 6(a), the external I/O module holds no dependency on any concrete adapter class, rendering the DIP enforcement described above directly visible. Behaviorally, the initialization phase of Fig. 6(b) realizes Algorithms 1 and 2 as interactions among these classes, whereas the operational phase reduces to retrieving a registered handle and invoking `send()` and `receive()` on it, confirming that no binding logic remains on the critical execution path.

4.3. Zero-code adaptability

The combination of dependency inversion, topology-driven orchestration, and the polymorphic factory collectively eliminates the need for source modification when the deployment environment changes. A representative scenario is the transition from live hardware to digital twin simulation. Operational radar systems typically communicate with external devices through specialized protocols such as RDMA or serial, whereas PC-based simulators rely on standard UDP or TCP. In legacy systems, accommodating this mismatch requires invasive source modification and a full recompilation. Under the proposed framework, the same transition is achieved by editing the network topology descriptor C_{net} alone: the orchestrator parses the updated configuration, the factory instantiates the corresponding adapters, and the application layer continues to invoke `send()` and `receive()` on I_{abs} without any awareness of the protocol change. As a result, the exact same compiled binary executes in both the live hardware environment and the simulation environment. The same mechanism applies equally to any deployment variation — such as integrating a new sensor topology or replacing a communication link — requiring only an updated topology descriptor rather than invasive source modification.

This configuration-driven adaptability directly resolves C2: transport changes are absorbed entirely by the external topology descriptor, the application remains closed for modification while the I/O subsystem remains open for extension, and the full recompilation-reverification cycle that dominates maintenance costs in legacy MFR systems is structurally eliminated.

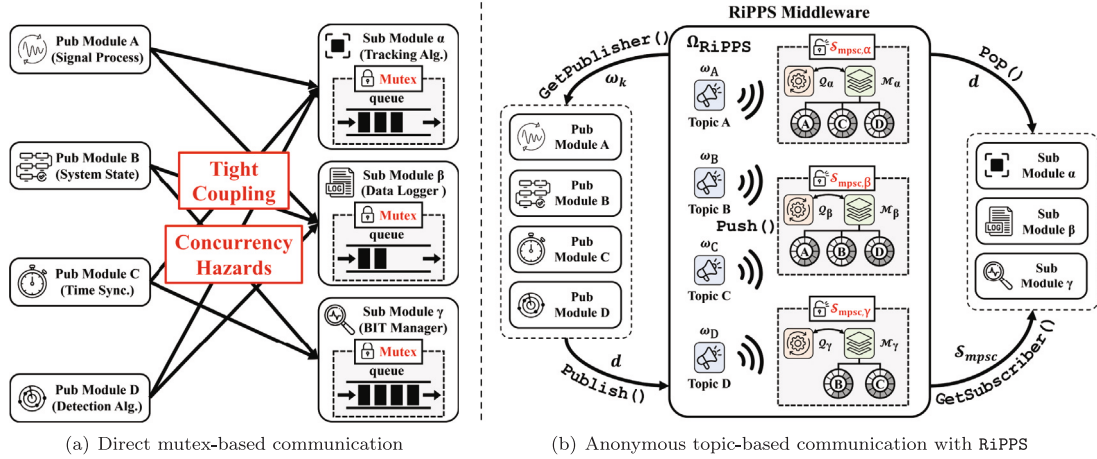


Fig. 7. Transition from legacy mutex-based communication to anonymous topic-based communication through RiPPS.

5. Intra-process communication with RiPPS

Building upon the macro-level layered architecture (Section 3) and the boundary-level dynamic I/O interface (Section 4), this section addresses the remaining micro-level challenge: the tight coupling and concurrency hazards in intra-process communication (C3). We first introduce an anonymous topic-based backbone that achieves architectural decoupling by replacing direct inter-module dependencies with a publish-subscribe topology. We then detail a memory-isolated multi-producer single-consumer (MPSC) stream that allocates a dedicated buffer to each producer thread, removing mutex involvement from the entire data transmission path. Together, these mechanisms constitute RiPPS, a lightweight intra-process middleware that structurally guarantees deadlock-free concurrency while preserving the architectural independence established at the preceding levels.

5.1. Anonymous topic-based backbone

The direct-reference coupling identified in C3, whose $M \times N$ dependency mesh is shown in Fig. 7(a), stems from the absence of an architectural intermediary to isolate senders from receivers. Replacing this coupling amounts to selecting among established messaging patterns — most prominently request-reply, point-to-point messaging, and publish-subscribe [19,20] — a choice recognized as a pivotal architectural decision when decomposing monolithic systems [21,22]. Although these patterns and their realizations have been extensively compared in microservice and IoT environments [23–26], the decisive criterion here is structural rather than performance-based. Request-reply requires each requester to address an identified replier in a one-to-one exchange; the 1:N and M:1 exchanges inherent to MFR pipelines would thus degenerate into enumerated pairwise sessions that retain the very direct references to be eliminated. Point-to-point messaging interposes a queue that decouples the exchange in time, yet each queue is addressed to a designated receiver, so a sender must still reference every queue it feeds—precisely the reference structure of the legacy model. Only publish-subscribe removes this mutual identification: the communicating parties exchange data exclusively through named topics, which decouples them in both space and synchronization [27], and a single topic natively accommodates multiple publishers and subscribers, providing the 1:N distribution and M:1 aggregation these pipelines demand.

RiPPS realizes this pattern by interposing an anonymous topic-based backbone between all communicating modules, as illustrated in Fig. 7(b). Instead of holding explicit references to a subscriber's input queue, modules obtain their communication handles solely from a central registry Ω_{RiPPS} . A publisher emits data to a named topic

Algorithm 3 Anonymous topic-based routing

```

1: Global State:
2:  $\Omega_{\text{RiPPS}}$ : Central registry mapping keys to channels
3:
4: // 1. Subscriber registration
5: function RiPPS::GetSubscriber(Key set  $\mathcal{K}$ )
6:    $S_{\text{mpsc}} \leftarrow \text{new StreamMPSC}()$ 
7:   for all Key  $k \in \mathcal{K}$  do
8:      $\omega_k \leftarrow \Omega_{\text{RiPPS}}.\text{GetTopicChannel}(k)$ 
9:      $\omega_k.\text{RegisterSubscriber}(S_{\text{mpsc}})$ 
10:  end for
11:  return  $S_{\text{mpsc}}$ 
12: end function
13:
14: // 2. Publisher registration
15: function RiPPS::GetPublisher(Key  $k$ )
16:    $\omega_k \leftarrow \Omega_{\text{RiPPS}}.\text{GetTopicChannel}(k)$ 
17:   return  $\omega_k$ 
18: end function
19:
20: // 3. Publish execution
21: function TopicChannel::Publish(Data  $d$ )
22:   for all Subscriber  $S_{\text{mpsc}} \in \text{this.SubscriberList}$  do
23:      $S_{\text{mpsc}}.\text{Push}(d)$ 
24:   end for
25: end function

```

channel, and a subscriber registers interest in one or more topics, with neither party aware of the other's identity. This indirection transforms the $M \times N$ dependency mesh into an event-driven topology in which every data path is mediated by the backbone.

Algorithm 3 formalizes this routing logic. The registry Ω_{RiPPS} maps unique topic keys to dedicated communication channels (Lines 1–2), and the overall communication proceeds in three phases:

- **Subscriber registration** (Lines 4–12): A subscriber often requires simultaneous input from heterogeneous sources—for example, a tracking algorithm subscribing to both signal plots and navigation updates. Rather than managing separate input queues for each source, the subscriber requests a handle for a set of topic keys $\mathcal{K}$. The middleware instantiates a single MPSC stream S_{mpsc} (whose internals are detailed in Section 5.2) and registers it with every corresponding topic channel ω_k . This single-stream-multiple-channel binding is the structural prerequisite for M:1 aggregation.
- **Publisher registration** (Lines 14–18): The publisher retrieves a reference to a specific topic channel ω_k without receiving any information about the subscribers attached to that channel. Because the channel

Algorithm 4 Memory-isolated MPSC stream

```

1: Internal Structure of  $S_{\text{mpsc}}$ :
2: // Data path: Isolated payload storage (wait-free)
3:  $\mathcal{M}_{\text{buf}}$ : Map<ThreadID, RingBuffer>
4: // Signal path: Global signal ordering (lock-free)
5:  $Q_{\text{sig}}$ : ConcurrentQueue<MetaData>
6:
7: // 1. Producer phase
8: function Push(Data  $d$ )
9: // ① Isolation: Acquire thread-local buffer
10:  $t_{\text{id}} \leftarrow \text{GetCurrentThreadID}()$ 
11: if  $\mathcal{M}_{\text{buf}}$ .Contains( $t_{\text{id}}$ ) is false then
12:    $\mathcal{M}_{\text{buf}}[t_{\text{id}}] \leftarrow \text{AcquireSPSC\_RingBuffer}()$ 
13: end if
14:  $\beta_{\text{local}} \leftarrow \mathcal{M}_{\text{buf}}[t_{\text{id}}]$ 
15: // ② Data path: Write payload to isolated memory
16:  $\beta_{\text{local}}.\text{Write}(d)$ 
17: // ③ Signal path: Notify availability to consumer
18:  $\mu_{\text{meta}} \leftarrow \{\text{ptr} : \beta_{\text{local}}, \text{size} : \text{Size}(d)\}$ 
19:  $Q_{\text{sig}}.\text{Enqueue}(\mu_{\text{meta}})$ 
20: end function
21:
22: // 2. Consumer phase
23: function Pop()
24:  $\mu_{\text{meta}} \leftarrow \text{null}$ 
25: // ④ Ordering: Poll signal queue for the next metadata
26: if  $Q_{\text{sig}}.\text{TryDequeue}(\mu_{\text{meta}})$  is true then
27:   // ⑤ Retrieval: Fetch from specific producer's buffer
28:    $\beta_{\text{src}} \leftarrow \mu_{\text{meta}}.\text{ptr}$ 
29:   return  $\beta_{\text{src}}.\text{Read}(\mu_{\text{meta}}.\text{size})$ 
30: end if
31: return  $\emptyset$ 
32: end function

```

encapsulates its own subscriber list, the publisher remains decoupled from all subscribers.

- *Publish execution* (Lines 20–25): Upon a publish event, the channel iterates through its subscriber list and pushes the payload d directly into each subscriber's stream via a memory copy. This single mechanism simultaneously realizes both routing patterns: 1:N distribution by fanning out d to every registered stream, and M:1 aggregation by funneling data from multiple channels into one unified input point. Because all participants reside within the same virtual address space, this multicast bypasses the serialization and marshaling stages that network-oriented middleware would require, ensuring deterministic latency suitable for high-frequency radar control loops.

5.2. Memory-isolated MPSC stream

While the anonymous topic-based backbone resolves architectural coupling, the underlying intra-process data exchange must guarantee thread safety without introducing deadlocks. Throughout this section, we adopt the *producer-consumer* terminology to describe the low-level queuing mechanism. To accurately characterize our design, we distinguish two levels of non-blocking guarantees: a *wait-free* operation completes within a bounded number of steps regardless of the actions of other threads, whereas a *lock-free* operation permits momentary contention — an individual thread may retry when concurrent accesses overlap — but guarantees that at least one thread always makes progress, preventing system-wide indefinite postponement or deadlocks.

Traditional approaches fuse data payload transfer and synchronization signaling into a single mutex-protected queue, forcing every producer to serialize on a single lock. This fusion is the structural root cause of the concurrency hazards identified in C3: circular lock acquisition chains across multiple queues create deadlock risks, and a

low-priority lock holder delays higher-priority producers, causing priority inversion. These hazards become particularly critical in a typical MFR processing pipeline dominated by the M:1 aggregation pattern, where multiple sensor and algorithm threads concurrently feed results into a single downstream consumer. Unlike the 1:N distribution pattern, in which a single producer sequentially dispatches data to independent consumer streams without inter-producer contention, this aggregation path forces multiple producers to converge on a single destination, forming the primary source of contention in the pipeline.

To address this contention at its structural origin, RiPPS employs a memory-isolated MPSC stream that physically separates the *data path* from the *signal path*, as visualized in Fig. 7(b). For the data path, the stream allocates a dedicated single-producer single-consumer (SPSC) ring buffer for each producer thread. This per-producer allocation eliminates inter-producer contention entirely, as no two producers ever access the same buffer, at the cost of a memory footprint growing with the number of communicating producers, analyzed in Section 6.3. Within each buffer, the SPSC design ensures that the sole writer and sole reader operate on structurally disjoint head and tail regions without requiring atomic compare-and-swap (CAS) instructions; consequently, both write and read operations achieve wait-free semantics. For the signal path, replicating signal queues per producer would not only force the sequentially processing consumer into inefficient multi-queue polling but also destroy the global arrival order of messages from independent producers. To preserve this global order, all producers share a single lock-free concurrent queue into which they enqueue lightweight metadata—comprising only a buffer pointer and a payload size. This separation structurally confines the unavoidable CAS contention to the signal path, where each contended operation carries only the lightweight metadata rather than a full radar payload that can span several kilobytes. The heavy payload transfer thus proceeds entirely on the wait-free data paths, ensuring deterministic performance even under high fan-in topologies.

Algorithm 4 details the logic governing this stream. The internal structure comprises a map of thread-local SPSC ring buffers $\mathcal{M}_{\text{buf}}$ for isolated payload storage and a shared concurrent queue Q_{sig} for global signal ordering (Lines 2–5). The stream operates in two phases:

- *Producer phase* (Lines 7–20): The Push function allows multiple producers to inject data concurrently. The algorithm first identifies the calling thread t_{id} and acquires a dedicated SPSC ring buffer if one has not yet been allocated (①). Because each buffer remains strictly exclusive to a single producer, this allocation enforces the physical memory isolation that underpins the wait-free data path. The producer then writes the payload d directly into this isolated buffer β_{local} via a wait-free memory operation (②). Finally, to notify the consumer, the producer enqueues the lightweight metadata packet μ_{meta} into the shared signaling queue Q_{sig} via a lock-free operation (③).
- *Consumer phase* (Lines 22–32): The Pop function retrieves data strictly following the global arrival order preserved by Q_{sig} . The consumer first polls the signaling queue via a lock-free dequeue to obtain the metadata of the next available message (④). Upon successful retrieval, this metadata identifies the specific source buffer β_{src} , from which the consumer extracts the payload via a wait-free read (⑤). If no metadata is available, the function immediately returns $\emptyset$ without blocking.

Fig. 8 presents the design of RiPPS in the same two aspects: the class diagram in Fig. 8(a) depicts its structure, and the sequence diagram in Fig. 8(b) its operation. Structurally, each topic channel realizes the Mediator pattern [18], encapsulating within a single class how publishers and subscribers interact. Fig. 8(a) also documents the internal structure of each participant, from the subscriber list of each channel to the thread-local SPSC ring buffers, the shared signaling queue, and the lightweight metadata inside the MPSC stream. In Fig. 8(a), no

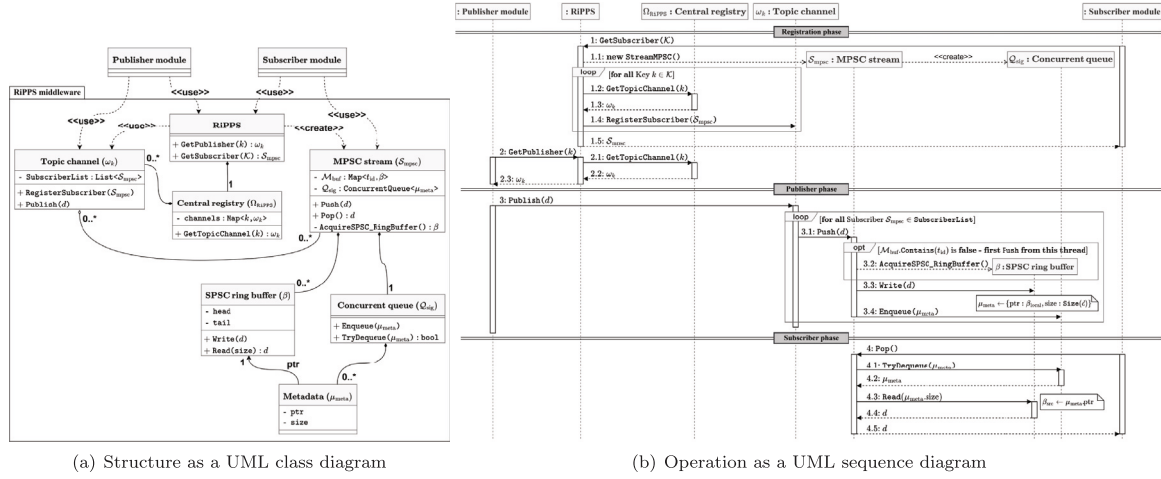


Fig. 8. Design of RiPPS.

dependency connects the publisher module to the subscriber module — the anonymity established above holds by construction — and the many-to-many association between topic channels and MPSC streams is the structural counterpart of the 1:N distribution and M:1 aggregation that RiPPS realizes. Behaviorally, Fig. 8(b) realizes Algorithms 3 and 4 as interactions among these classes across its registration, publisher, and subscriber phases, with steps ①–⑤ of Algorithm 4 appearing as messages 3.2–3.4, 4.1, and 4.3. The object lifecycles in Fig. 8(b) further visualize where the isolation originates: the MPSC stream and its signaling queue are created once at subscriber registration, whereas each thread-local ring buffer is acquired only upon the first Push() from its producer, extending the memory isolation as producers join.

This memory-isolated mechanism provides two structural safety guarantees that directly resolve the concurrency hazards identified in C3. First, by entirely excluding mutexes from the data transmission path, the architecture eliminates the *circular wait* condition required for deadlocks by design. Second, the wait-free semantics of the data path ensure that the payload transfer of a high-priority thread is never delayed by a preempted lower-priority thread, structurally preventing priority inversion on this path. Ultimately, by synergizing the anonymous decoupling of the topic-based backbone with this memory-isolated stream, RiPPS uniquely achieves both architectural independence and deadlock-free concurrency within a single lightweight intra-process middleware.

6. Evaluation

This section evaluates the proposed framework by measuring reductions in complexity and coupling after applying all three architectural strategies to the target MFR subsystems, and by benchmarking the concurrency properties of RiPPS against representative alternatives.

6.1. Methodology

Target systems. We evaluated the proposed framework on two operational MFR subsystems provided by a defense industry partner, both extracted from the deployed software stack of a modern phased array radar: a radar control system (RCS) and an antenna control system (ACS). The RCS implements mission-critical functions including multi-target tracking, beam scheduling, and resource management [28–30], constituting the central control software of the MFR. The ACS receives real-time control commands from the RCS to operate antenna components, performing beam steering, antenna calibration, and status monitoring [31,32], while interfacing with numerous external antenna

assemblies through diverse I/O protocols. While the RCS thus commands the ACS at runtime, the two are separate software subsystems within a single MFR—an MFR comprises multiple such subsystems, each developed and deployed independently. Because the functional requirements of these subsystems differ fundamentally, a common software framework targets a single subsystem across the radar variants of the product family. We accordingly applied the proposed framework to the two subsystems independently, yielding a separate restructured baseline for each. Each reduction reported in this section thus compares one subsystem’s legacy monolithic implementation with the restructured baseline of the same subsystem. We selected the RCS and the ACS because, within an MFR software stack, they carry the largest codebases, the most diverse functionality, and the most extensive external device interfaces, and therefore constitute the most demanding restructuring targets. These two subsystems differ substantially in scale, role, and hardware coupling intensity; evaluating both verifies that the proposed framework yields consistent architectural improvements across heterogeneous subsystem profiles rather than over-fitting to a single profile.

Structural metrics. To quantify the architectural impact of the restructuring, we collected five code-level metrics across the entire codebase of each subsystem before and after applying all three strategies. For structural complexity, we measured MCC, LOC, and raw pointer usage count, which reflects the burden of manual resource management and implicit coupling through shared memory references. For inter-module coupling, we measured Fan-in/out as defined in Eq. (3), which counts the number of distinct modules referencing or referenced by a given module, and TCR-in/out, which accumulates the total reference count across module boundaries. The former is analogous to the coupling between objects (CBO) metric [33], which quantifies the breadth of inter-class dependencies; TCR-in/out extends this notion by capturing their intensity. A module may have a Fan-out of 3, yet a TCR-out of 30 implies that the coupling is concentrated on those few dependencies, creating a structural bottleneck that Fan-in/out alone can miss.

Concurrency benchmark. To isolate the concurrency safety of RiPPS under controlled conditions, we conducted measurements on a server equipped with an Intel Xeon D-2796NT CPU @ 2.00 GHz and 64 GB RAM, comparing four intra-process communication strategies:

- StdMtx (std::mutex + std::queue): the legacy baseline representing the direct-coupling model identified in C3, where each producer explicitly references a consumer’s queue under mutual exclusion.

Table 1

Structural complexity and coupling metrics of each subsystem before and after its framework adoption.

Radar Control System (RCS)			
Metric	Before	After	Δ
MCC	12,061	5,944	−51%
LOC	261,017	113,698	−56%
Pointer count	2,897	892	−69%
Fan-in/out	642	314	−51%
TCR-in/out	5,110	2,948	−42%
Antenna Control System (ACS)			
Metric	Before	After	Δ
MCC	569	435	−24%
LOC	42,921	33,729	−21%
Pointer count	244	53	−78%
Fan-in/out	173	142	−18%
TCR-in/out	1,212	994	−18%

- **BoostLF** (`boost::lockfree::queue`): a lock-free alternative that eliminates blocking but retains direct inter-module references, isolating the effect of lock removal without addressing structural coupling.
- **SimDDS**: a full-stack DDS baseline that faithfully reproduces the five-layer middleware overhead of a standard DDS — UDP loopback transport, dynamic endpoint discovery, mutex-guarded history cache, quality-of-service (QoS) policy checking, and common data representation (CDR) serialization — to quantify the cost of applying a network-oriented publish-subscribe stack to intra-process communication.
- **RiPPS** (proposed): the hybrid MPSC stream with anonymous topic-based routing operating entirely in user space, as described in Section 5.

We evaluated these strategies under $N \times N$ mesh topologies scaling from 2×2 to 32×32 to assess scalability under increasing producer contention. Producers emitted 1 kB payloads — representing typical MFR intra-process messages such as target plots and beam steering commands — at randomized intervals across three pacing configurations. The baseline interval of 500–1000 μ s represents typical MFR search-and-track cycles, while the progressively tighter intervals of 200–500 μ s and 100–200 μ s were specifically designed to stress-test each strategy under extreme publish rates. StdMtx, BoostLF, and RiPPS were configured with an identical per-queue capacity of 65,536 payload slots, all three retrying a failed push after a yield; SimDDS instead queued its in-flight messages in the kernel socket buffers of its UDP loopback transport and kept its history caches bounded.

For each configuration, we executed 50 independent Monte Carlo simulations with a 1-second warmup phase and a 5-second measurement window per run; the two highest and two lowest throughput outliers were discarded to reduce transient variance. We report three latency metrics: average latency for steady-state transmission cost, P99.9 tail latency for worst-case real-time safety, and latency jitter for timing predictability.

6.2. Structural improvement

Structural complexity. Table 1 summarizes the five metrics measured across the entire codebase of each subsystem before and after restructuring. The RCS exhibited an MCC reduction of 51%, from 12,061 to 5944. This reduction indicates that modules previously consolidating cross-layer responsibilities — such as interleaving tracking logic with protocol dispatch and synchronization management — were decomposed into layer-specific units with simpler control flows through the combined effect of all three strategies. LOC decreased by 56%, from 261,017 to 113,698; the layered decomposition

Table 2

Architectural property comparison of intra-process communication strategies.

Property	StdMtx	BoostLF	SimDDS	RiPPS
Anonymity	×	×	✓	✓
Deadlock-free	×	✓	× ^a	✓
Priority-inversion-free	×	×	×	✓ ^b
Zero-kernel overhead	✓	✓	×	✓

^a SimDDS history cache requires mutex for concurrent access.

^b Data path only; signal path uses lock-free CAS.

enabled systematic reuse of common framework-layer utilities, reducing the protocol-handling and synchronization logic that had been independently maintained across legacy modules. The raw pointer count dropped by 69%, from 2897 to 892. This reduction is consistent with the structural changes introduced across the framework: the layered architecture enforces unidirectional dependency flow, preventing lower-layer modules from referencing upper-layer data structures; the dynamic I/O interface replaces device-specific pointer references with polymorphic abstract handles; and the anonymous topic-based backbone of RiPPS eliminates direct inter-module references entirely, removing the need for pointer-based data exchange between modules.

The ACS showed comparatively moderate reductions in MCC (−24%) and LOC (−21%), consistent with its smaller pre-restructuring baseline: a codebase of approximately 43,000 LOC inherently offers fewer opportunities for structural simplification than one exceeding 261,000 LOC. However, the ACS achieved a 78% reduction in pointer count—the largest relative improvement across both subsystems. This outcome aligns with the ACS’s architectural role: as the subsystem directly interfacing with numerous external antenna assemblies through diverse communication protocols, its legacy implementation relied heavily on raw pointers for device communication and internal message exchange, making it particularly receptive to the pointer-eliminating mechanisms of the dynamic I/O interface and RiPPS.

Coupling and dependency. The aggregate Fan-in/out sum — capturing the total number of inter-module dependency relationships across each subsystem — decreased by 51% in the RCS and 18% in the ACS. While the dependency metric D derived from Fan-in and Fan-out served as the quantitative classification criterion for layer assignment (Section 3), these aggregate reductions confirm that the proposed framework successfully reduced the overall structural entanglement that the V - D -based decomposition was designed to address. TCR-in/out decreased by 42% in the RCS and 18% in the ACS, indicating that the restructuring not only reduced the number of coupled modules but also lowered the reference density at each remaining dependency. This coupling reduction reflects the convergent effect of all three strategies: the layered architecture restricts the directions along which references can propagate, the dynamic I/O interface decouples business logic from concrete protocol implementations, and RiPPS replaces direct inter-module queue references with an anonymous topic-based backbone.

While the ACS reductions are smaller in magnitude, they are consistent in direction across all five metrics. The difference in reduction margins between the two subsystems reflects a difference in pre-restructuring entanglement rather than a limitation of the framework; a less entangled codebase naturally yields narrower reduction margins when the same restructuring principles are applied.

6.3. Concurrency benchmark

Architectural properties. Before examining quantitative performance, we compare the four strategies in terms of the architectural properties required to resolve C3. Table 2 summarizes four properties: communication anonymity, deadlock freedom, priority-inversion freedom, and zero-kernel overhead. StdMtx satisfies only zero-kernel overhead; modules must hold direct references to each other’s queues,

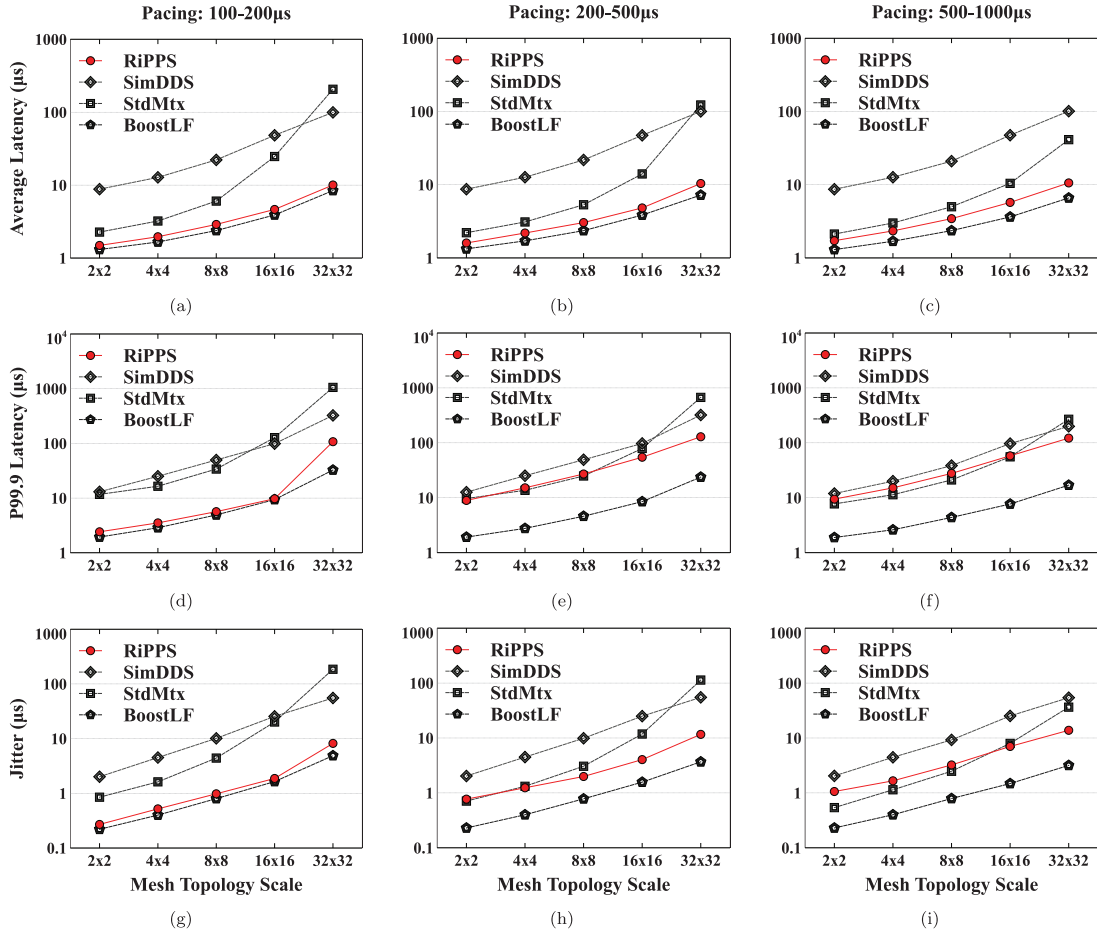


Fig. 9. Concurrency benchmark results: average latency, P99.9 tail latency, and jitter under varying pacing intervals and mesh topologies.

and mutual exclusion introduces both deadlock risk and priority inversion. BoostLF eliminates deadlock by removing locks from the data path, but it retains direct inter-module references and remains susceptible to priority inversion: because its CAS-based queue requires producers to retry upon contention [34], a high-priority thread can be repeatedly delayed by lower-priority threads winning CAS races. This susceptibility highlights the necessity of wait-free semantics to guarantee priority-inversion freedom [35]. SimDDS achieves communication anonymity through its publish-subscribe model, yet its reliance on UDP loopback transport and mutex-guarded history caching reintroduces kernel transitions and blocking hazards, forfeiting both zero-kernel efficiency and deadlock freedom. RiPPS is the only strategy that simultaneously satisfies all four properties: the anonymous topic-based backbone ensures communication anonymity, the memory-isolated MPSC stream with wait-free producer semantics guarantees deadlock freedom and priority-inversion freedom on the data path, and the pure user-space implementation eliminates kernel overhead.

Latency and scalability. Fig. 9 presents the average latency, P99.9 tail latency, and jitter measured under $N \times N$ mesh topologies across the three pacing configurations, spanning from the 500–1000 μs range representing typical MFR operation to the 100–200 μs stress test. Under the typical pacing in Fig. 9(c), RiPPS sustained an average latency from 1.72 μs at 2×2 to 10.58 μs at 32×32 . This sub-11 μs envelope held robustly even under the most demanding 100–200 μs pacing in Fig. 9(a), scaling from 1.49 to 10.04 μs . In contrast, SimDDS exhibited 5–10 $\times$ higher average latency across all topologies and pacing configurations. Notably, at 32×32 , its average latency remained nearly invariant at 99–101 μs across Fig. 9(a)–(c), whereas RiPPS varied only within 10.04–10.58 μs . This pacing invariance confirms that the dominant cost originates from the full middleware stack — serialization, discovery,

and kernel transitions — which imposes a fixed overhead independent of pacing pressure, while the scaling with topology density reflects kernel-level network stack contention under high-density communication. This result quantitatively validates RiPPS’s design decision to develop a dedicated intra-process middleware rather than repurpose a network-oriented DDS.

RiPPS’s structural immunity to lock contention becomes most evident against the mutex-based baseline. Under the typical 500–1000 μs pacing, StdMtx remained competitive at moderate scales (recording 5.00 μs versus RiPPS’s 3.44 μs at 8×8 in Fig. 9(c)) but degraded to 41.24 μs at 32×32 , whereas RiPPS sustained 10.58 μs . This 4 $\times$ gap in average latency extended to a 2.2 $\times$ gap in P99.9 tail latency (268.78 versus 121.81 μs in Fig. 9(f)) and a 2.6 $\times$ gap in jitter (36.47 versus 13.80 μs in Fig. 9(i))—levels already incompatible with the timing predictability required for high-density MFR operation. Crucially, this degradation intensified super-linearly as the publish rate increased: at the intermediate 200–500 μs pacing, the 32×32 average-latency gap widened to 12 $\times$ (Fig. 9(b)), and under the tightest 100–200 μs pacing, StdMtx collapsed to 206.33 μs at 32×32 (a 20 $\times$ gap in Fig. 9(a)), with its P99.9 tail latency exploding to 1055.56 μs (Fig. 9(d)) and jitter surging to 185.56 μs (Fig. 9(g)). This systemic collapse directly validates the concurrency hazards identified in C3: lock contention grows non-linearly with both the number of producers sharing a mutex-guarded queue and the publish rate, rendering mutex-based data exchange structurally unsuitable for MFR intra-process communication.

RiPPS performed competitively with BoostLF, the raw-latency leader across all conditions. At 32×32 under the typical 500–1000 μs pacing, RiPPS recorded an average latency of 10.58 μs versus BoostLF’s 6.60 μs in Fig. 9(c), maintaining this sub-4 μs gap across all pacing configurations. The P99.9 tail latency was nearly identical

at moderate scales (9.73 versus 9.38 μs at 16×16 under the tightest 100–200 μs pacing in Fig. 9(d)), with the divergence appearing at 32×32 . However, this divergence reveals a stability characteristic: across Fig. 9(d)–(f), RiPPS's P99.9 remained robustly bounded within a predictable 107–129 μs envelope, whereas BoostLF's tail latency degraded from 16.96 to 32.60 μs as the publish interval tightened. This behavior is an expected consequence of RiPPS's two-phase memory-isolated stream design (Section 5.2): each publish operation performs a wait-free payload write into a dedicated SPSC ring buffer followed by a lock-free metadata enqueue into a shared signal queue. Although BoostLF faces comparable per-queue CAS contention under the same 32×32 mesh, it completes each publish in a single CAS step. In contrast, RiPPS incurs the structural cost of both phases, with the signal-queue contention from 32 simultaneous producers amplifying the tail—a pattern similarly reflected in the jitter metrics across Fig. 9(g)–(i). Crucially, this structural cost is pacing-independent by construction, bounding RiPPS's P99.9 envelope regardless of the underlying publish pressure.

Memory overhead. Because the wait-free data path requires each producer to own an exclusive buffer, every subscriber's stream holds one SPSC ring buffer per producer thread that feeds it: under the 32×32 mesh, RiPPS allocates 32 ring buffers per stream — 1024 in total — whereas StdMtx and BoostLF serve each consumer through a single shared queue. Under the identical per-queue capacity of Section 6.1, RiPPS therefore reserves 32 times the buffer capacity that either baseline assigns to a consumer, and in exchange confines each producer's bursts to its own ring instead of headroom shared with the others. This reservation remains an idle margin rather than a performance advantage, because no queue of the three operates near its capacity: by Little's law [36], a queue's backlog is bounded by its own arrival rate multiplied by its own latency. The shared StdMtx queue receives at most 0.32 messages per microsecond — 32 producers, each publishing no faster than once per 100 μs — and its 1055.56 μs P99.9 in Fig. 9(d), the largest of the benchmark, bounds its backlog at $0.32 \times 1055.56 \approx 338$ messages, about 0.5% of its capacity; each RiPPS ring, fed by a single producer at no more than 0.01 messages per microsecond, holds fewer than two messages even at the top of the 107–129 μs P99.9 envelope reported above.

The configured capacity thus isolates the measurement from buffer exhaustion rather than prescribing a deployment size: an operational deployment provisions each ring far more tightly, and its total footprint follows from the publish–subscribe topology at design time—the number of connected producer–subscriber pairs multiplied by the per-ring capacity and the payload size; the full mesh evaluated here is the densest such topology. Conversely, splitting the baselines into per-producer queues would equalize the footprint and relieve their contention, yet still provide no anonymity; the memory profile is therefore a consequence of the wait-free data path, not the source of the architectural properties of Table 2.

Architecture versus latency. The benchmark results reveal a clear tradeoff between architectural properties and raw latency. BoostLF delivers the lowest latency because it bypasses the routing indirection entirely—producers write directly into target queues, avoiding both the topic resolution and the two-phase publish overhead required by RiPPS. However, as Table 2 demonstrates, this latency advantage comes at the cost of forfeiting communication anonymity and priority-inversion freedom—precisely the properties required to resolve C3. In contrast, RiPPS incurs a modest routing overhead to achieve full anonymity and wait-free producer guarantees, while maintaining average latency within 0.2–4 μs of BoostLF and remaining one order of magnitude below SimDDS across all tested conditions. For MFR intra-process communication, where the primary design objective is to enforce the architectural boundaries established in Sections 3–5 without reintroducing inter-module coupling, this tradeoff is not merely acceptable but architecturally necessary: the sub-11 μs average latency that RiPPS sustains even at 32×32 is well within the operational

timing budgets of radar signal processing pipelines, and the deadlock-free, priority-inversion-free guarantees on the data path cannot be achieved by any of the alternative strategies, regardless of their raw latency advantages.

7. Related work

In this section, we critically review general module decomposition methodologies together with the architectural paradigms, abstraction techniques, and communication middleware prevailing in defense embedded systems, and analyze why these approaches remain insufficient to address the challenges C1–C3.

7.1. Gaps in module decomposition methodologies

How a software system should be partitioned into modules has been answered through a succession of decomposition criteria. Functional decomposition derives module boundaries from the processing steps of the system [37], data-driven decomposition aligns the program structure with the structure of the data it processes [38], and object-oriented decomposition, rooted in information hiding [39], organizes the system around encapsulated objects [40]. Subsequent methodologies broadened the criteria: aspect-oriented decomposition extracts cross-cutting concerns such as synchronization and logging into separately specified aspects [41], domain-driven design partitions the system into bounded contexts identified through collaborative domain modeling [42], and event-driven decomposition identifies modules from the events they produce and consume, as recently practiced in monolith-to-microservices migration [43]. Notably, information hiding already placed change at the center of this question, prescribing that each module conceal a design decision likely to change. Under every one of these criteria, drawing the module boundary — and, in particular, anticipating what will change — remains an act of foresight by the architect.

Transforming a legacy codebase into a common software framework for a product family, however, poses a question of observation rather than foresight: the modules already exist, and so do the variants; what must be determined is which modules have remained invariant across them. None of the above criteria makes this property visible, because a module cohesive under any of them can mix code that survives every variant with code that each variant rewrites; a functionally cohesive tracking module, for instance, may combine invariant filter mathematics with engagement doctrine tuned to each deployment. Indeed, the legacy subsystems restructured in this work are written in an object-oriented language, yet exhibit the uncontrolled coupling of C1: encapsulation localizes a change behind a class interface, but does not indicate which classes remain invariant across the product family. Recent automated decomposition approaches replace this foresight with quantitative clustering [43], yet the static structure, runtime behavior, and version history they operate on are all observed within the single system under decomposition, whereas volatility is observable only through cross-project comparison.

The proposed V – D -based decomposition does not displace these methodologies; within each layer of the resulting architecture, fine-grained module boundaries can continue to follow functional or object-oriented practice. What it contributes is the decision they leave to foresight: it operationalizes the change-centered criterion of information hiding by replacing anticipated change with change measured across the variants of the product family, and pairs this measured volatility with each module's dependency within the system under restructuring, as described in Section 3.2. Once the analysis scope is declared, the classification of a legacy module thus follows from evidence recorded in the code rather than from a priori judgment, and feeds directly into the layer assignment of Section 3.3. None of the surveyed methodologies derives its module classification from measured cross-project change, and for a legacy product family — where the evidence of what has actually changed already exists — this is what makes the reusable core assets identifiable in the first place.

7.2. Gaps in architectural decomposition for internal modularity

The defense community has widely adopted open architecture standards such as MOSA [2] and SOSA [3] to mitigate vendor lock-in and improve interoperability. These initiatives successfully standardize the external interfaces between subsystems by defining hardware slot profiles, connector specifications, and data exchange formats, yet they deliberately treat the internal software structure as a black box, providing no guidance on how to decompose legacy monolithic systems into granular, independently reusable modules with governed dependency directions. A system can therefore achieve full standards compliance while retaining a structurally entangled internal organization—a condition we term *compliance without modularity*.

While these standards govern external interfaces, SPLE manages variability across product variants by enabling engineers to select specific features for a given configuration [4,5]. SPLE and modularization techniques are complementary: SPLE determines what to reuse at the feature level, whereas how the underlying code modules should be structurally organized rests with the product-line architect. Thus, fulfilling this responsibility on a legacy defense codebase requires an objective criterion for judging which modules qualify as reusable core assets and where the architectural boundaries should be drawn. Moreover, to guarantee that these assets are structurally independent enough to be extracted without pulling in unrelated dependencies, the architect must quantitatively distinguish the two orthogonal architectural dimensions that govern reusability: module volatility and module dependency.

At the code-architecture level, Martin's package design principles — notably the SDP and the stable abstractions principle (SAP) [16] — provide a sound theoretical foundation by prescribing that dependency should flow toward stability. However, these principles remain general-purpose heuristics that define the desirable properties of a well-structured system but do not specify how to quantitatively assess each module's volatility and dependency in an existing legacy codebase, nor how to map those assessments onto a concrete layered architecture with an enforced unidirectional dependency flow [44]. Software reference architectures likewise consolidate proven structural knowledge for reuse across the systems of a domain [45], yet instantiating such an architecture on a legacy codebase presupposes precisely this unresolved step: determining which existing modules qualify for which layer of the target structure. To date, no prior work has proposed a module decomposition methodology that classifies defense radar software modules along these two orthogonal metrics and maps them onto dependency-governed layers, leaving C1 unresolved.

7.3. Gaps in runtime I/O adaptability under static binding

Decoupling application logic from hardware specifics is typically achieved via a HAL [6]. In general-purpose computing, dynamic polymorphism is widely used to support runtime flexibility for such abstraction. In contrast, embedded defense systems predominantly rely on compile-time static binding — such as conditional compilation directives and preprocessor macros — for their HAL implementations, driven by a longstanding concern over runtime overhead. This prevailing concern, however, is fundamentally flawed, as it conflates two distinct operational phases: initialization and execution. Runtime binding through an IoC container occurs exclusively during the system initialization phase, where a configuration file is parsed and protocol-specific objects are instantiated and injected before any real-time operation begins. Once the system transitions to its operational phase, all bindings are fully resolved, incurring zero additional overhead on the critical execution path.

Despite this misconception, the prevailing reliance on static binding persists in practice, forcing invasive source-code modifications and full recompilation whenever the communication protocol changes — whether replacing UDP with RDMA for a new sensor topology or

switching from hardware-bound protocols to standard LAN interfaces for digital twin simulation — directly violating the OCP. Although DIP and IoC are well-established techniques in mainstream software engineering, their adoption for I/O abstraction in embedded defense systems remains limited. Conventional IoC frameworks [9] such as Spring presuppose managed runtime environments such as the JVM and encapsulate heavyweight infrastructure — including bean lifecycle management, aspect-oriented proxying, and transaction coordination — that far exceeds the minimal injection mechanism required for I/O protocol binding, making their adoption impractical on resource-constrained defense embedded platforms.

Modern embedded ecosystems organize their software design around abstraction layers as well, yet their binding mechanisms follow two directions, neither of which lifts these constraints. In the first, the configuration that drives the abstraction is consumed before compilation: the classic AUTOSAR platform, dominant in automotive control units, generates its component-binding code from an offline configuration [46], so a transport change still culminates in regeneration and recompilation. In the second, the binding is resolved dynamically but only within a full middleware stack: ROS 2 interposes a vendor-neutral interface that exchanges the underlying DDS implementation without modifying application code [7], the adaptive AUTOSAR platform binds services atop a POSIX operating system [8], and micro-ROS extends this stack to microcontrollers through a client-agent architecture [47]. Each of these mechanisms presupposes adopting the respective ecosystem's runtime and infrastructure wholesale, reproducing at the ecosystem scale the footprint barrier identified above for conventional IoC frameworks. Meanwhile, existing embedded HAL practices continue to hard-code the binding between business logic and concrete protocol drivers, lacking a mechanism to resolve protocol selection at runtime from an external topology descriptor. No existing approach provides a lightweight, configuration-driven IoC mechanism that enables zero-code protocol adaptability — including seamless transitions between live hardware and digital twin simulation environments — within these constraints, leaving C2 unresolved.

7.4. Gaps in intra-process communication decoupling and safety

The publish-subscribe model defined by the DDS standard offers a principled alternative to the direct-reference coupling identified in C3, introducing anonymous, topic-based communication that eliminates explicit inter-module dependencies [10]. However, standard DDS implementations are architected for distributed network environments. When applied to high-frequency intra-process control loops, they impose the full middleware stack overhead — including CDR serialization and deserialization, participant discovery protocol maintenance, and user-kernel context switches for loopback transport — that is prohibitive for the microsecond-scale determinism required by radar beam scheduling and tracking loops [11–13].

Recent efforts have sought to mitigate this overhead from different angles. Eclipse iceoryx [48] achieves near-zero-copy inter-process communication through shared-memory transport, yet its reliance on a central daemon and atomic reference counting introduces infrastructural complexity and cache coherency overheads that are justified for inter-process isolation but disproportionate for lightweight intra-process decoupling within a single address space.

At the data-structure level, high-performance lock-free queues such as `boost::lockfree::queue` [14] and `moodycamel::ConcurrentQueue` [49] deliver excellent concurrent throughput. However, they remain low-level synchronization primitives that structurally force threads to share explicit pointers to specific queue instances. They provide no publish-subscribe routing and no topic-based anonymity; consequently, integrating them into a communication architecture still requires developers to manually wire producers to consumers, reintroducing the very coupling that motivated the search for middleware. To date, no existing solution combines the architectural

anonymity of the DDS publish–subscribe model with a lightweight, lock-free concurrency mechanism within a single intra-process middleware — one that simultaneously guarantees structural independence and deadlock-free execution without network stack overhead — leaving C3 unresolved.

8. Discussion and limitations

The evaluation in Section 6 substantiates the intended benefits of the framework; each of its guarantees, however, carries a cost. The V - D -based layered architecture requires each legacy subsystem to undergo a one-time refactoring; the zero-code adaptability of the dynamic I/O interface moves the detection of a misconfigured topology descriptor from compile time to initialization, though still before any real-time operation begins; and the anonymity and wait-free producer guarantees of RiPPS are bought with a bounded latency and memory cost over a bare lock-free queue. Because the refactoring cost is paid once per subsystem, the resulting structural improvement accrues to the subsystem itself, independent of any subsequent variant; what recurs with every variant derived from the restructured baseline is the return from cross-project reuse. The framework therefore realizes its full value in a product-family context.

Three limitations bound these findings. First, both evaluated subsystems were extracted from the software stack of a single deployed phased array radar; their heterogeneous profiles support, but do not by themselves establish, generality beyond this product family. Second, measuring volatility presupposes source snapshots of the existing variants in the declared analysis scope; the framework therefore targets legacy product families in which such evidence has accumulated, and offers no substitute for architectural foresight in a first-of-a-kind development. Third, the anonymity and deadlock freedom of RiPPS hold within a single process; exchanges crossing process or node boundaries traverse the dynamic I/O interface, whose endpoints the topology descriptor explicitly identifies—a division consistent with the per-subsystem scope of the common framework.

Finally, the source code of the framework and the restructured subsystems originates from an operational defense product and cannot be publicly released for security reasons. The paper instead specifies each mechanism at a reproducible level of detail: Algorithms 1–4 define the orchestration, factory, routing, and stream logic, Figs. 6 and 8 document their structure and operation in UML, and Section 6.1 reports the complete benchmark configuration, enabling independent reimplementations of the mechanisms and verification of the concurrency results.

9. Conclusion

This paper addresses the fundamental challenge of establishing a reusable software baseline for diverse MFR product families whose legacy codebases are constrained by monolithic structural entanglement. We presented a common software framework that resolves inter-module coupling across the macro, boundary, and micro architectural levels, enabling structural flexibility and systematic reusability across heterogeneous radar configurations. The proposed framework integrates three synergistic architectural strategies: (i) a volatility-dependency-based layered architecture that enforces a strict unidirectional dependency flow to structurally isolate invariant core assets from volatile project-specific modifications; (ii) a protocol-agnostic dynamic I/O interface, driven by an inversion-of-control mechanism, that achieves zero-code adaptability to heterogeneous deployment environments; and (iii) RiPPS, a lightweight intra-process publish–subscribe middleware that guarantees anonymous communication and structurally deadlock-free concurrency via a memory-isolated stream mechanism. Evaluation on two operational MFR subsystems demonstrated that this integrated restructuring yields reductions of up to 51% in modified cyclomatic complexity, 42% in total coupling references, and

56% in lines of code. Furthermore, comparative concurrency benchmarks confirmed that RiPPS uniquely preserves the communication anonymity of a publish–subscribe architecture while eliminating the full-stack network overhead that renders standard DDS impractical for intra-process deployment. Future work includes extending this framework to multi-node distributed radar architectures and exploring automated architectural compliance verification tools to govern dependency constraints throughout continuous product-line evolution.

CRedit authorship contribution statement

Jinsoo Park: Writing – original draft, Visualization, Validation, Software, Methodology, Investigation, Formal analysis, Conceptualization. **Heejeong Hong:** Validation, Software, Resources, Investigation. **Ikjun Yeom:** Writing – review & editing. **Jinkyu Lee:** Writing – review & editing, Supervision, Project administration, Funding acquisition.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work was supported by the National Research Foundation of Korea (NRF) grant funded by the Korea government (MSIT) (RS-2024-00438248, RS-2025-22502968). This research was also supported by the Yonsei University Research Fund of 2025 (2025-22-0408).

Data availability

The authors do not have permission to share data.

References

- [1] M.A. Richards, J. Scheer, W.A. Holm, W.L. Melvin, *Principles of Modern Radar*, SciTech Publishing, Inc., 2010.
- [2] P. Zimmerman, M. Ofori, D. Barrett, J. Soler, A. Harriman, Considerations and examples of a modular open systems approach in defense systems, *J. Def. Model. Simul.* 16 (4) (2019) 373–388.
- [3] C.P. Collier, I. Lipkin, S.A. Davidson, J. Dirner, Sensor open system architecture (SOSA), in: *Open Architecture/Open Business Model Net-Centric Systems and Defense Transformation 2016*, Vol. 9849, SPIE, 2016, 984903.
- [4] K. Pohl, G. Böckle, F. Van Der Linden, *Software Product Line Engineering: Foundations, Principles, and Techniques*, Springer, 2005.
- [5] P. Clements, L. Northrop, *Software Product Lines*, Addison-Wesley, 2002.
- [6] K. Popovici, A. Jerraya, *Hardware abstraction layer: Introduction and overview*, in: *Hardware-Dependent Software: Principles and Practice*, Springer, 2009, pp. 67–94.
- [7] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, W. Woodall, Robot operating system 2: Design, architecture, and uses in the wild, *Sci. Robot.* 7 (66) (2022) eabm6074.
- [8] C. Menard, A. Goens, M. Lohstroh, J. Castrillon, Achieving determinism in adaptive AUTOSAR, in: *2020 Design, Automation & Test in Europe Conference & Exhibition, DATE, IEEE*, 2020, pp. 822–827.
- [9] M. Fowler, Inversion of control containers and the dependency injection pattern, 2004, <https://martinfowler.com/articles/injection.html>.
- [10] G. Pardo-Castellote, OMG data-distribution service: Architectural overview, in: *Proceedings of the 23rd International Conference on Distributed Computing Systems Workshops, IEEE*, 2003, pp. 200–206.
- [11] G. Sciangula, D. Casini, A. Biondi, C. Scordino, M. Di Natale, Bounding the data-delivery latency of DDS messages in real-time applications, in: *Proceedings of Euromicro Conference on Real-Time Systems, ECRTS*, 2023, pp. 9:1–9:26.
- [12] V. Bode, C. Trinitis, M. Schulz, D. Buettner, T. Preclik, DDS implementations as real-time middleware – A systematic evaluation, in: *Proceedings of IEEE International Conference on Embedded and Real-Time Computing Systems and Applications, RTCSA*, 2023, pp. 186–195.
- [13] X. Luo, X. Jiang, H. Liang, Y. Tang, N. Guan, W. Yi, Flexible zero-copy IPC for processing chains in ROS 2, *IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst.* 45 (8) (2026) 3985–3999.

- [14] T. Blechmann, Boost.Lockfree C++ Library, <https://www.boost.org/doc/libs/latest/doc/html/lockfree.html>.
- [15] R.C. Martin, Agile Software Development, Principles, Patterns, and Practices, Prentice Hall, 2003.
- [16] R.C. Martin, Clean Architecture: A Craftsman's Guide to Software Structure and Design, Prentice Hall Press, 2017.
- [17] T. Górski, Software architecture description in original software publications, *Softw. Impacts* 27 (2026) 100802.
- [18] E. Gamma, R. Helm, R. Johnson, J. Vlissides, Design Patterns: Elements of Reusable Object-Oriented Software, Addison-Wesley, 1995.
- [19] G. Hohpe, B. Woolf, Enterprise Integration Patterns: Designing, Building, and Deploying Messaging Solutions, Addison-Wesley Professional, 2004.
- [20] I.K. Aksakalli, T. Çelik, A.B. Can, B. Tekinerdoğan, Deployment and communication patterns in microservice architectures: A systematic literature review, *J. Syst. Softw.* 180 (2021) 111014.
- [21] M. Waseem, P. Liang, A. Ahmad, M. Shahin, A.A. Khan, G. Márquez, Decision models for selecting patterns and strategies in microservices systems and their evaluation by practitioners, in: Proceedings of the 44th International Conference on Software Engineering: Software Engineering in Practice, 2022, pp. 135–144.
- [22] J. Kazanavičius, D. Mažeika, Evaluation of microservice communication while decomposing monoliths, *Comput. Inform.* 42 (1) (2023) 1–36.
- [23] B. Çiftçi, B. Çiloğlu, A review of comparative studies on performance evaluation of communication mechanisms for microservices, in: International Conference on Computational Science and its Applications, Springer, 2025, pp. 98–113.
- [24] R. Bux, G.S. Shenoy, Performance analysis of RESTFUL web services and RABBITMQ for microservices based systems on cloud environment, in: 2024 3rd International Conference for Innovation in Technology, INOCON, IEEE, 2024, pp. 1–6.
- [25] M. Niswar, R.A. Safruddin, A. Bustamin, I. Aswad, Performance evaluation of microservices communication with REST, GraphQL, and gRPC, *Int. J. Electron. Telecommun.* 70 (2) (2024) 429–436.
- [26] A. Shahrokhi, M. Ahmadi, Power evaluation of IOT application layer protocols, in: 2023 7th International Conference on Internet of Things and Applications, IoT, 2023, pp. 1–7.
- [27] P.T. Eugster, P.A. Felber, R. Guerraoui, A.-M. Kermarrec, The many faces of publish/subscribe, *ACM Comput. Surv.* 35 (2) (2003) 114–131.
- [28] P.W. Moo, Z. Ding, Adaptive Radar Resource Management, Elsevier, 2015.
- [29] A. Charlish, F. Hoffmann, C. Degen, I. Schlangen, The development from adaptive to cognitive radar resource management, *IEEE Aerosp. Electron. Syst. Mag.* 35 (6) (2020) 8–19.
- [30] J. Park, H. Park, Y. Cho, J. Lee, RT-Radar: Maneuver-aware real-time beam scheduling for multi-target tracking tasks, in: 2026 IEEE 32nd Real-Time and Embedded Technology and Applications Symposium, RTAS, 2026, pp. 316–329.
- [31] S. Jang, J. Kim, S. Kim, C. Cheon, Implementation of electronic beam steering system using look-up table calibration method, *Microw. Opt. Technol. Lett.* 56 (12) (2014) 2769–2773.
- [32] J.S. Herd, M.D. Conway, The evolution to modern phased array architectures, *Proc. IEEE* 104 (3) (2016) 519–529.
- [33] S.R. Chidamber, C.F. Kemerer, A metrics suite for object oriented design, *IEEE Trans. Softw. Eng.* 20 (6) (1994) 476–493.
- [34] A. Morrison, Y. Afek, Fast concurrent queues for x86 processors, in: Proceedings of the 18th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming, 2013, pp. 103–112.
- [35] C. Yang, J. Mellor-Crummey, A wait-free queue as fast as fetch-and-add, in: Proceedings of the 21st ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming, 2016, pp. 1–13.
- [36] J.D. Little, S.C. Graves, Little's law, in: Building Intuition: Insights from Basic Operations Management Models and Principles, Springer, 2008, pp. 81–100.
- [37] W.P. Stevens, G.J. Myers, L.L. Constantine, Structured design, *IBM Syst. J.* 13 (2) (1974) 115–139.
- [38] M.A. Jackson, Principles of Program Design, Academic Press, Inc., 1975.
- [39] D.L. Parnas, On the criteria to be used in decomposing systems into modules, *Commun. ACM* 15 (12) (1972) 1053–1058.
- [40] G. Booch, R.A. Maksimchuk, M.W. Engle, B.J. Young, J. Conallen, K.A. Houston, Object-Oriented Analysis and Design with Applications, third ed., Addison-Wesley, 2007.
- [41] G. Kiczales, J. Lamping, A. Mendhekar, C. Maeda, C. Lopes, J.-M. Loingtier, J. Irwin, Aspect-oriented programming, in: European Conference on Object-Oriented Programming, Springer, 1997, pp. 220–242.
- [42] E. Evans, Domain-Driven Design: Tackling Complexity in the Heart of Software, Addison-Wesley Professional, 2004.
- [43] Y. Abgaz, A. McCarren, P. Elger, D. Solan, N. Lapuz, M. Bivol, G. Jackson, M. Yilmaz, J. Buckley, P. Clarke, Decomposition of monolith applications into microservices architectures: A systematic review, *IEEE Trans. Softw. Eng.* 49 (8) (2023) 4213–4242.
- [44] Z. Wan, Y. Zhang, X. Xia, Y. Jiang, D. Lo, Software architecture in practice: Challenges and opportunities, in: Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, 2023, pp. 1457–1469.
- [45] L. Garcés, S. Martínez-Fernández, L. Oliveira, P. Valle, C. Ayala, X. Franch, E.Y. Nakagawa, Three decades of software reference architectures: A systematic mapping study, *J. Syst. Softw.* 179 (2021) 111004.
- [46] M. Staron, Automotive Software Architectures, Springer, 2021.
- [47] K. Belsare, A.C. Rodriguez, P.G. Sánchez, J. Hierro, T. Kolcon, R. Lange, I. Lütkebohle, A. Malki, J.M. Losa, F. Melendez, et al., Micro-ROS, in: Robot Operating System (ROS): The Complete Reference (Volume 7), Springer, 2023, pp. 3–55.
- [48] Eclipse Foundation, True Zero-Copy Inter-Process-Communication, <https://github.com/eclipse-iceoryx/iceoryx>.
- [49] C. Desrochers, A Fast Multi-Producer, Multi-Consumer Lock-Free Concurrent Queue For C++11, <https://github.com/cameron314/concurrentqueue>.